ENG23 2011
Database Systems

Indexing in Databases

Lecturer: Asst. Prof. Dr.Nuntawut Kaoungku AFHEA

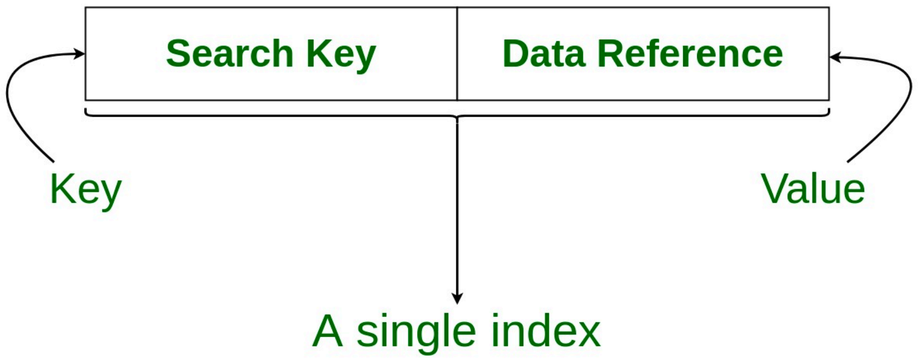
What is Indexing?

- Indexing is a way to optimize the performance of a database by minimizing the number of disk accesses required when a query is processed.
- It is a data structure technique which is used to quickly locate and access the data in a database.

What is Indexing?

- Indexes are created using a few database columns.
  - The first column is the Search key that contains a copy of the primary key or candidate key of the table. These values are stored in sorted order so that the corresponding data can be accessed quickly.
  - Note: The data may or may not be stored in sorted order.
  - The second column is the Data Reference or Pointer which contains a set of pointers holding the address of the disk block where that particular key value can be found.

Structure of an Index in Database



Types of Indexing in DBMS

There are mainly 4 types of indexing methods

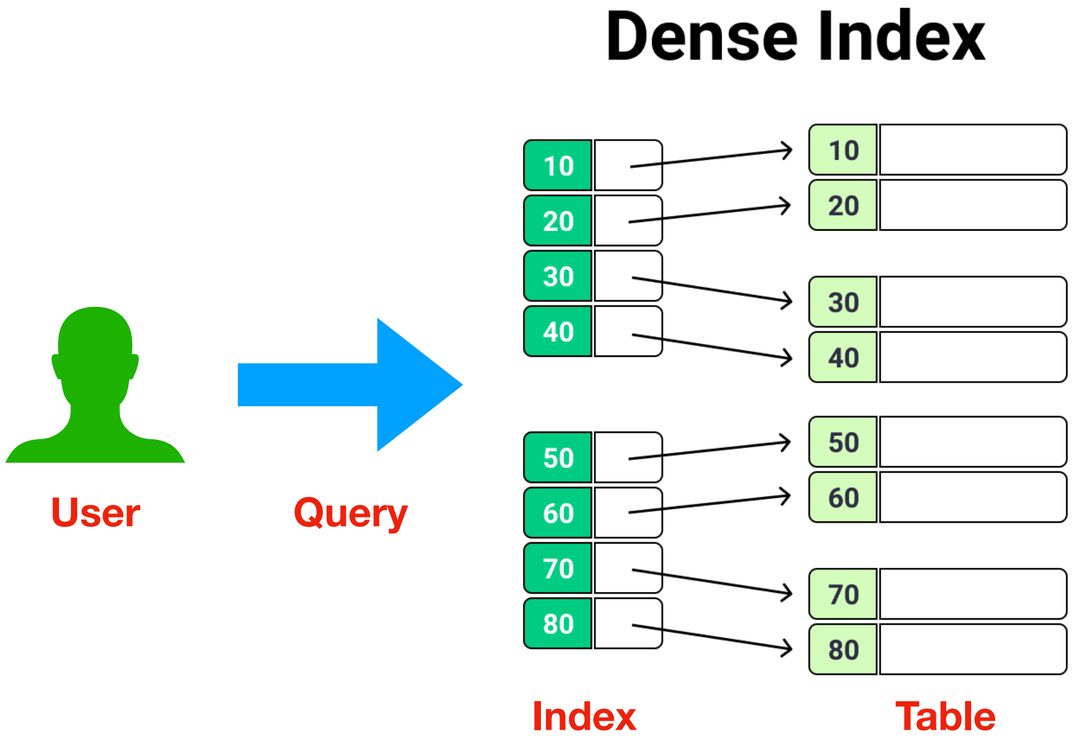
- Primary Indexing
- Secondary Indexing
- Cluster Indexing
- Multilevel Indexing

Primary Indexing

- Primary Index is an ordered file which has fixed length size with two fields.
- The first field is the same a primary key and second field is pointed to that specific data block.
- In the primary index, there is always one to one relationship between the entries in the index table.
- It is also known as a *clustering index*
- Primary Indexing is further divided into two types.
    - Dense Index
    - Sparse Index

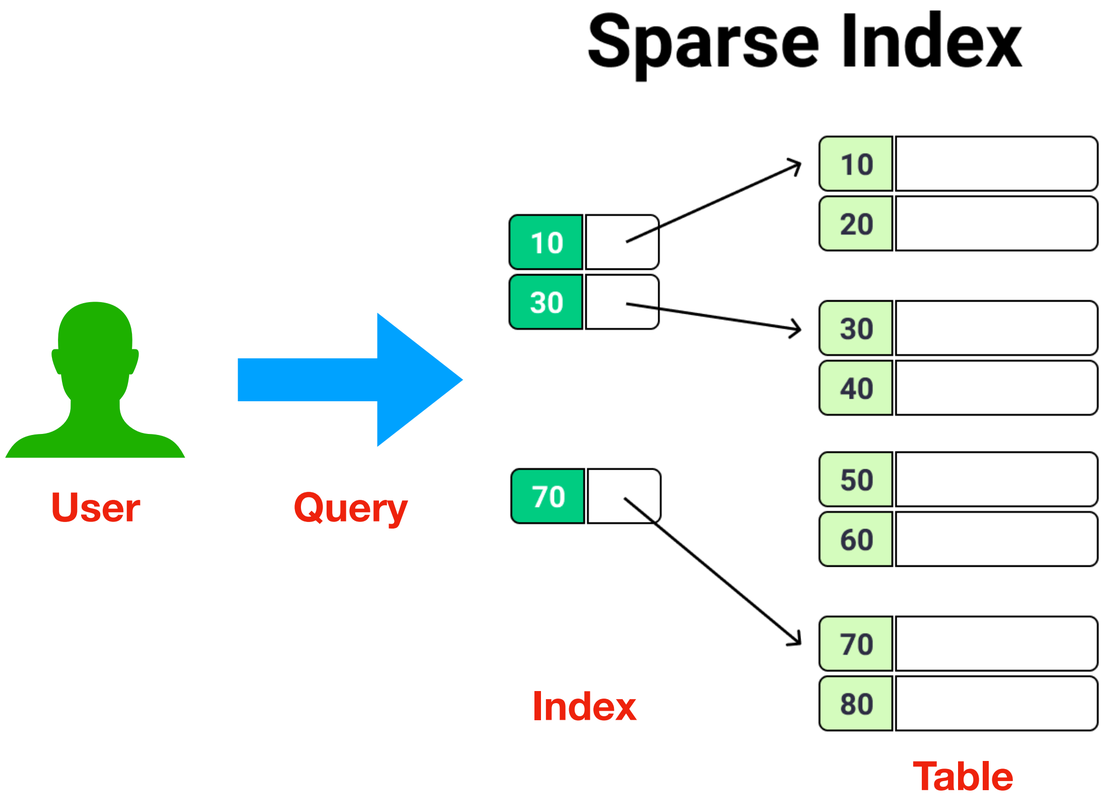
Dense Index

- In a dense index, a record is created for every search key valued in the database.
- Dense indexing helps you to search faster *but needs more space to store index records.*
- In dense indexing, records contain search key value and points to the real record on the disk.



Sparse Index

- The sparse index is an index record that appears for only some of the values in the file.
- Sparse Index helps you to resolve the issues of dense indexing.
- In sparse indexing technique, a range of index columns stores the same data block address, and when data needs to be retrieved, this block address will be fetched.
- Sparse indexing method stores index records for only some search key values.
- It needs less space, less maintenance overhead for insertion, and deletions *but it is slower compared to the dense index for locating records.*

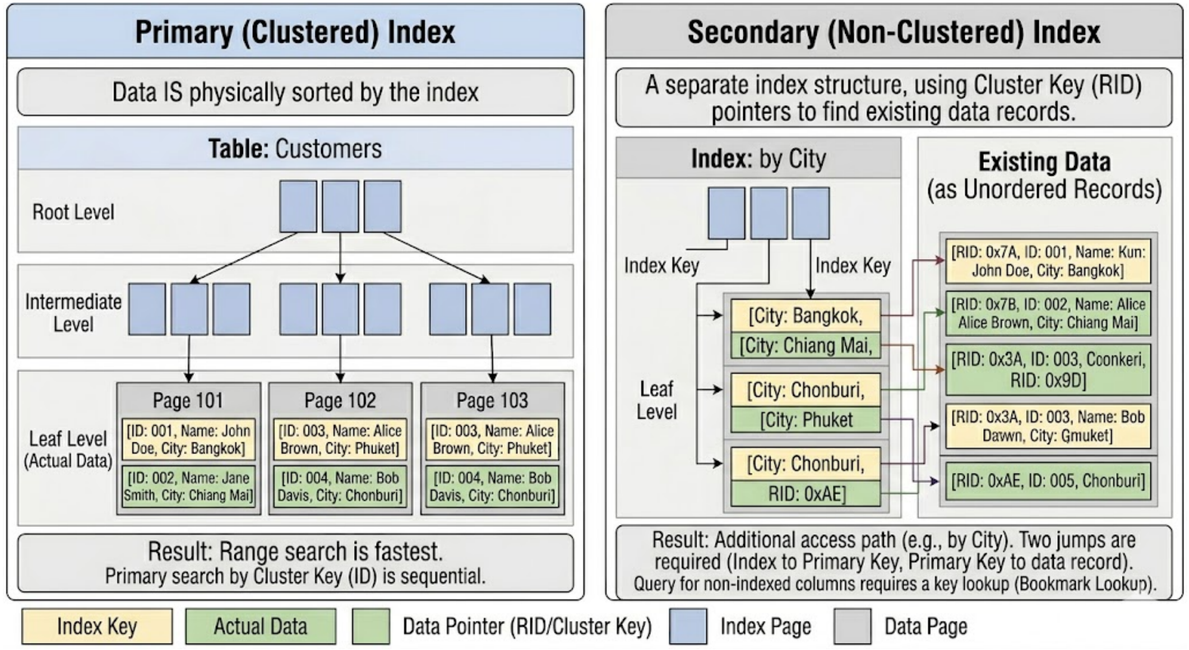


Clustering Index

- A clustered index can be defined as an ordered data file. Sometimes the index is created on non-primary key columns which may not be unique for each record.

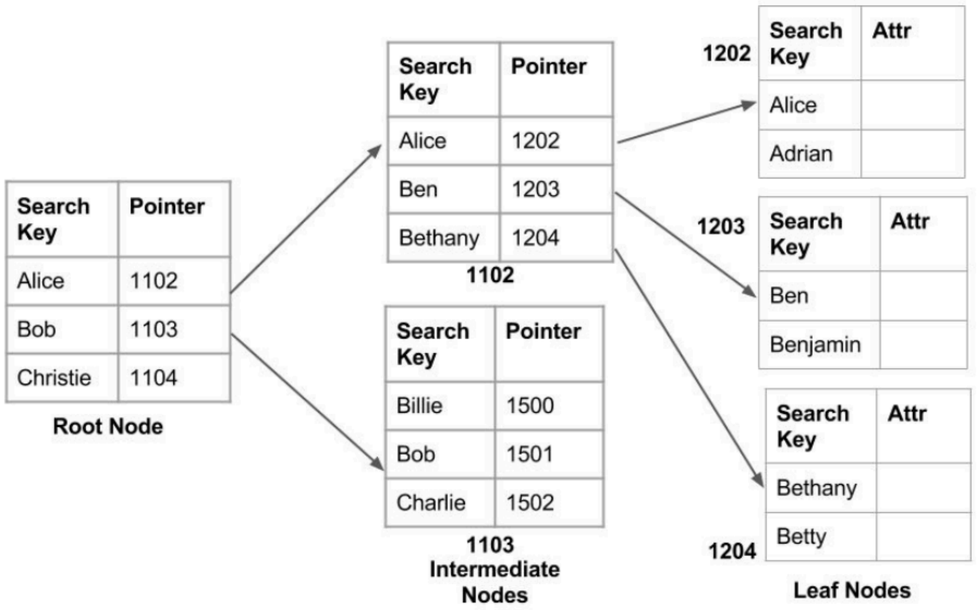
- In this case, to identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called a clustering index.

- The records which have similar characteristics are grouped, and indexes are created for these group.



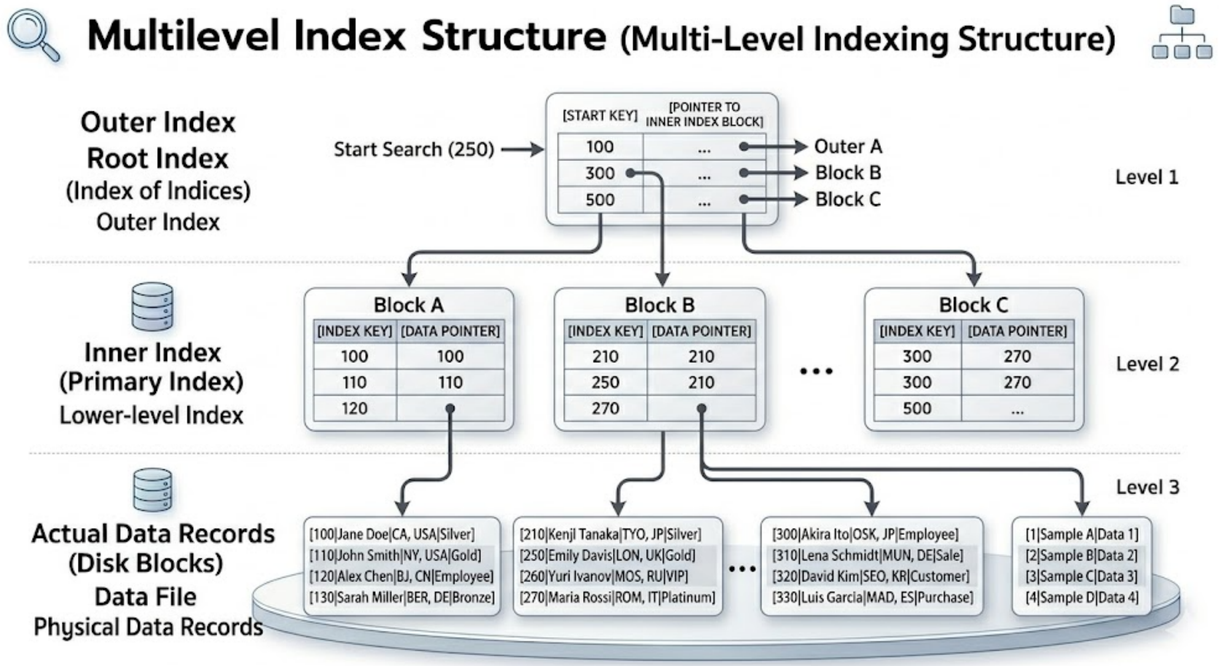
Secondary Indexing

- The secondary index can be generated by a field which has a unique value for each record.
- It is also known as a *non-clustering index*.
- This two-level database indexing technique is used to *reduce the mapping size of the first level.*
- For the first level, a large range of numbers is selected, because of this mapping size always remains small.



Multilevel Indexing

- Multilevel Indexing is created when a primary index does not fit in memory.
- In this type of indexing method, you can reduce the number of disk accesses to short any record and kept on a disk as a sequential file and create a sparse base on that file.

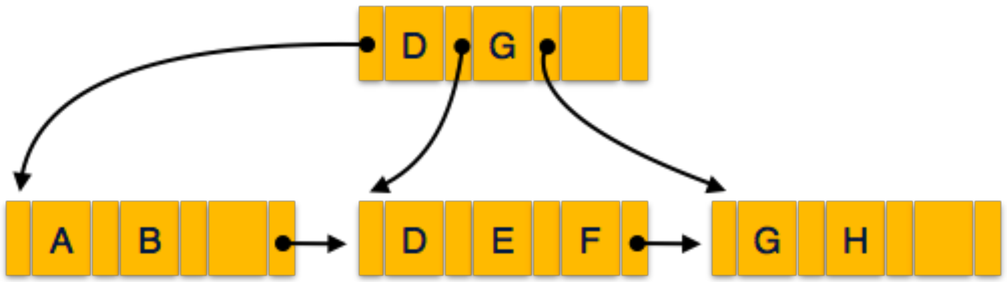


B+ Tree

- The B+ tree is a balanced binary search tree. It follows a multi-level index format.
- In the B+ tree, leaf nodes denote actual data pointers. B+ tree ensures that all leaf nodes remain at the same height.
- In the B+ tree, the leaf nodes are linked using a link list. Therefore, a B+ tree can support random access as well as sequential access.

Structure of B+ Tree

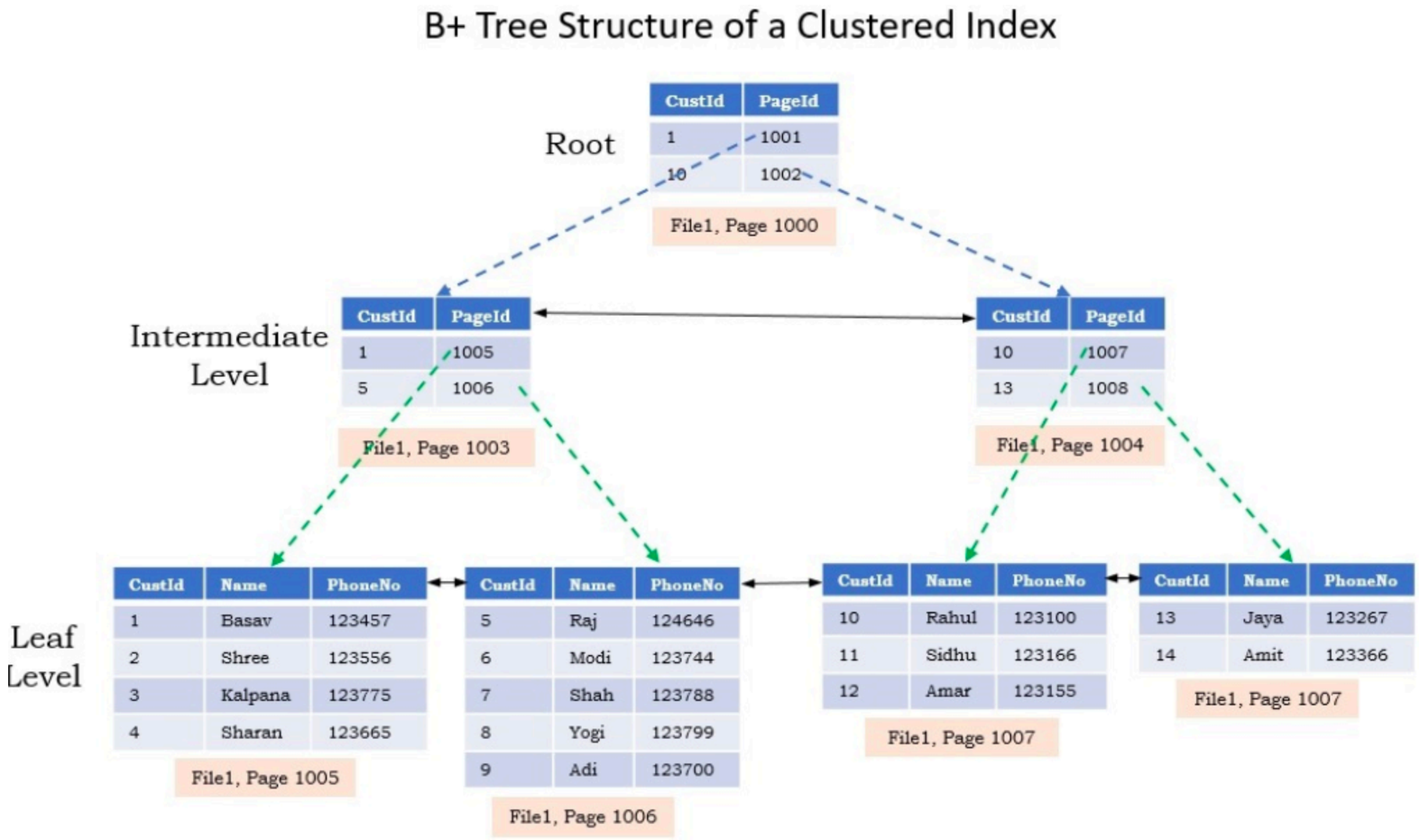
- Every leaf node is at equal distance from the root node. A B+ tree is of the order n where n is fixed for every B+ tree.
- Internal nodes –
    - Internal (non-leaf) nodes contain at least ⌈n/2⌉ pointers, except the root node.
    - At most, an internal node can contain n pointers.
- Leaf nodes –
    - Leaf nodes contain at least ⌈n/2⌉ record pointers and ⌈n/2⌉ key values.
    - At most, a leaf node can contain n record pointers and n key values.
    - Every leaf node contains one block pointer P to point to next leaf node and forms a linked list.



Searching a record in B+ Tree

- Perform a binary search on the records in the current node.
- If a record with the search key is found, then return that record.
- If the current node is a leaf node and the key is not found, then report an unsuccessful search.
- Otherwise, follow the proper branch and repeat the process.

Searching a record in B+ Tree

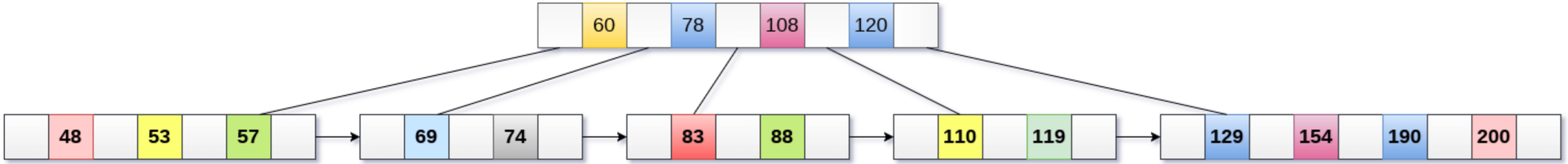


B+ Tree Insertion

- Step 1: Insert the new node as a leaf node
- Step 2: If the leaf doesn't have required space,
    - (1) split the node and
    - (2) copy the middle node to the next index node.
- Step 3: If the index node doesn't have required space,
    - (1) split the node and
    - (2) copy the middle element to the next index page.

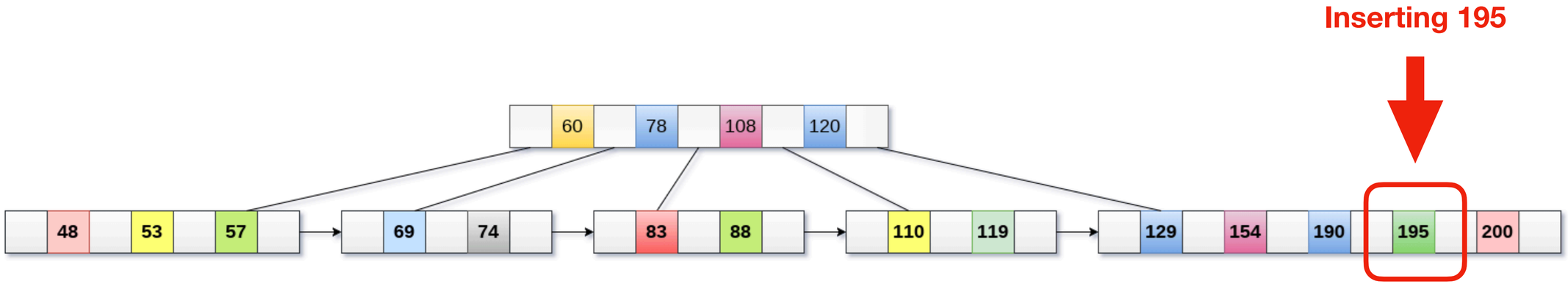
Example : Inserting 195

- Insert the value **195** into the B+ tree of order 5 shown in the following figure.



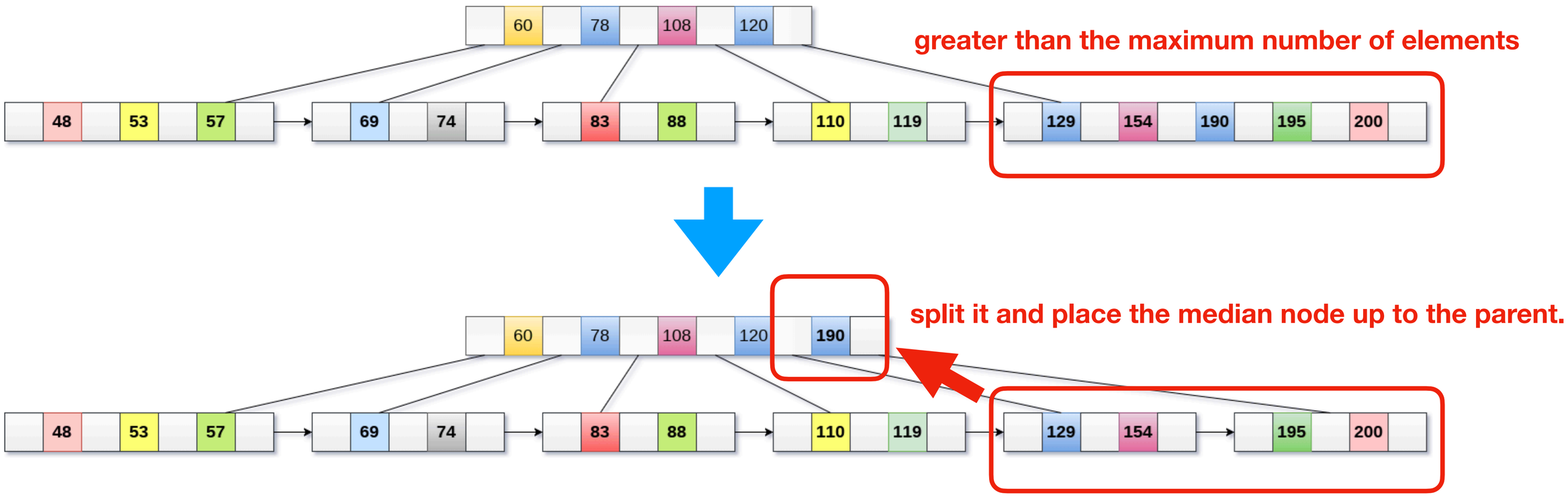
Example : Inserting 195

- 195 will be inserted in the right sub-tree of 120 after 190. Insert it at the desired position



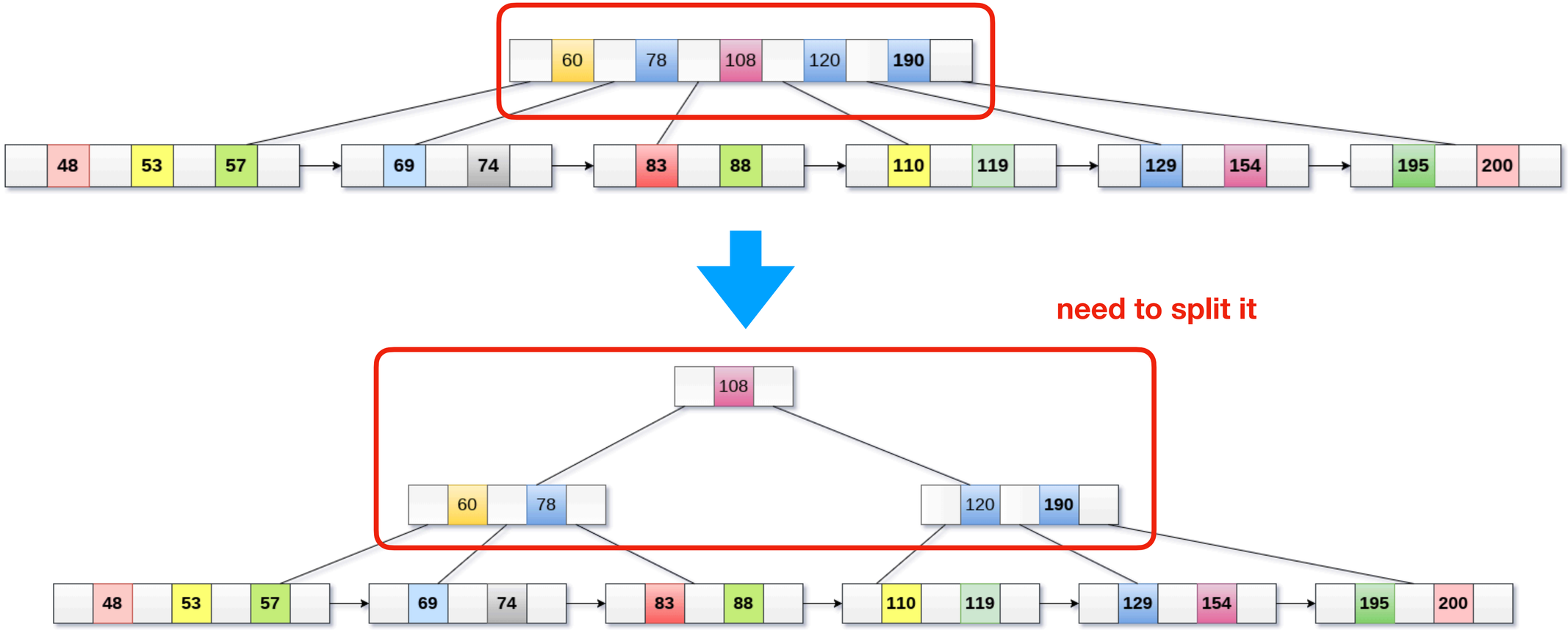
Example : Inserting 195

- The node contains greater than the maximum number of elements i.e. 4, therefore split it and place the median node up to the parent.



Example : Inserting 195

- Now, the index node contains 6 children and 5 keys which violates the B+ tree properties, therefore we need to split it, shown as follows.

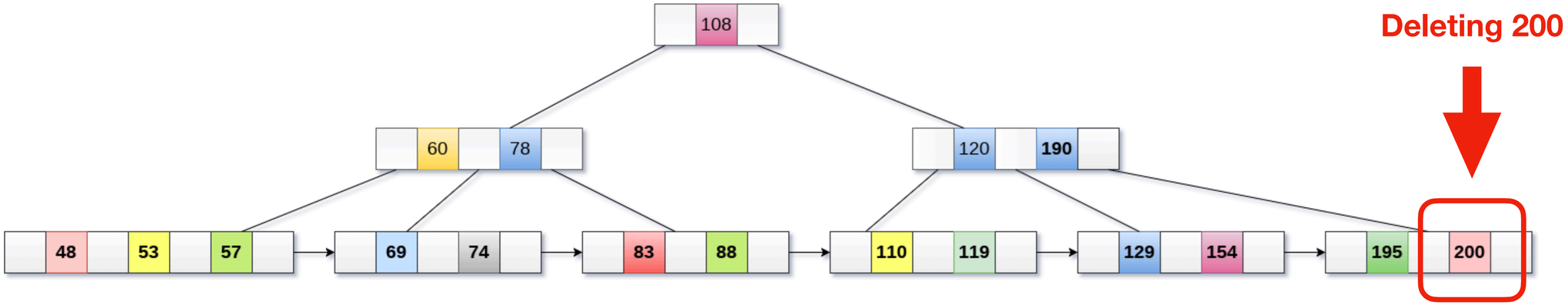


B+ Tree Deletion

- Step 1: Delete the key and data from the leaves.
- Step 2: if the leaf node contains less than minimum number of elements, merge down the node with its sibling and delete the key in between them.
- Step 3: if the index node contains less than minimum number of elements, merge the node with the sibling and move down the key in between them.

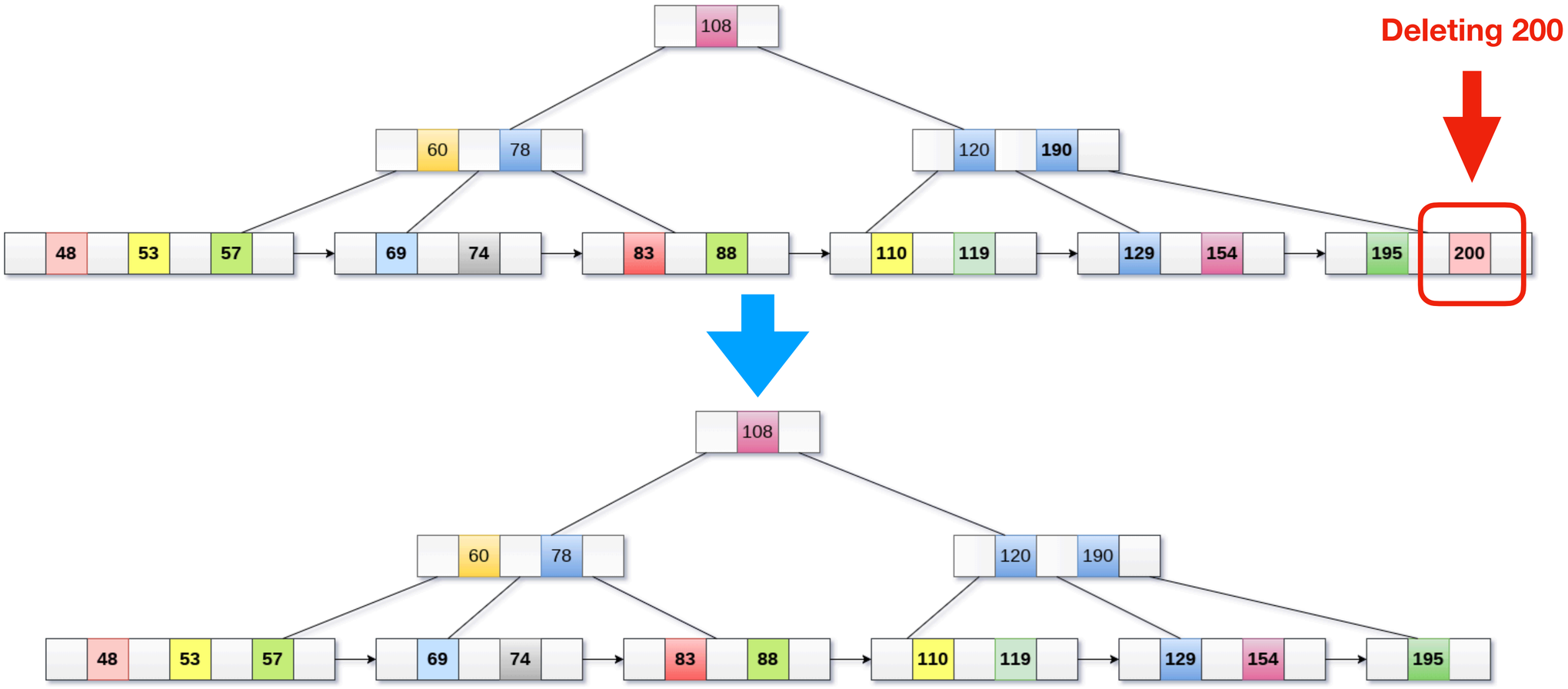
Example : Deleting 200

- Delete the key **200** from the B+ Tree shown in the following figure.



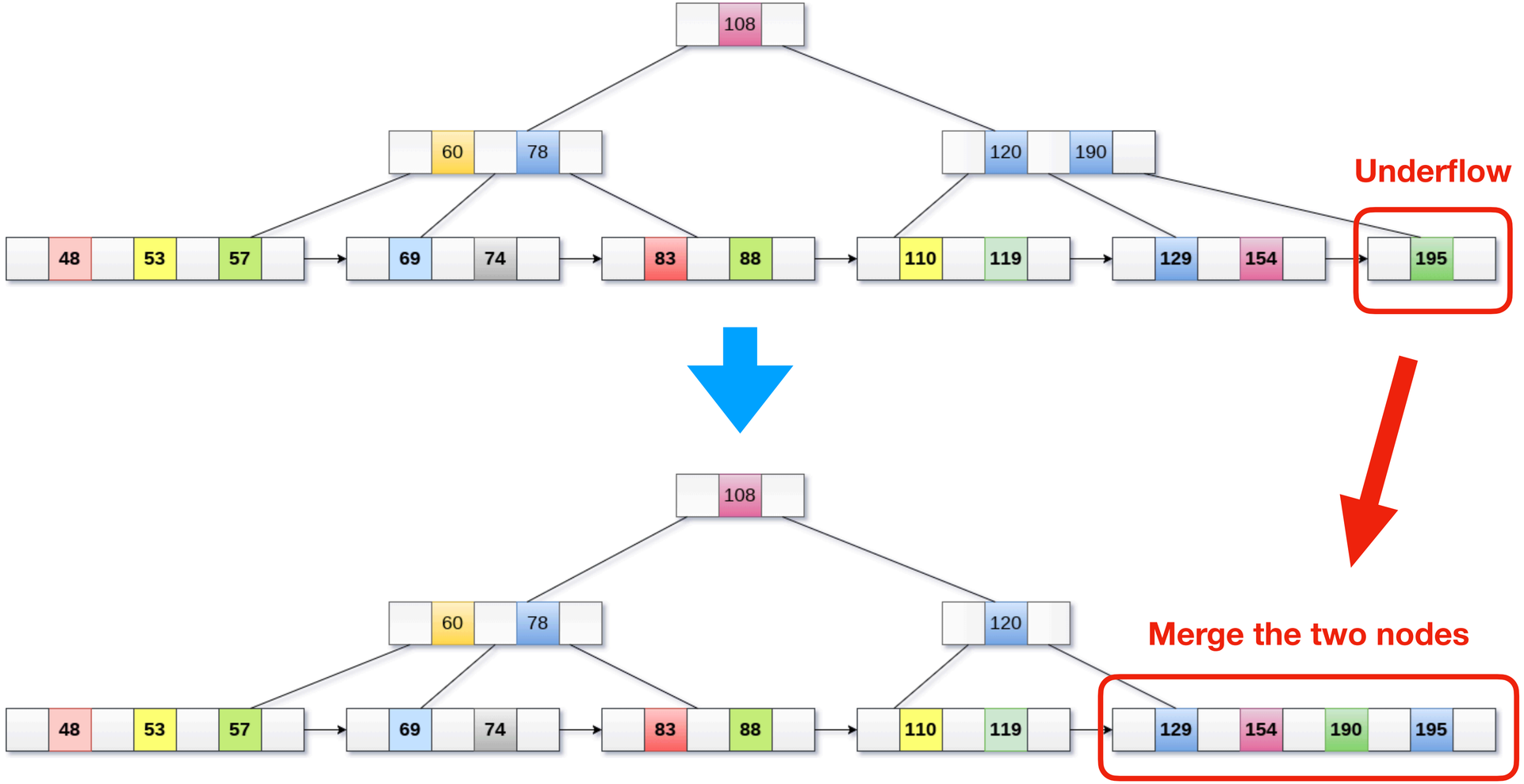
Example : Deleting 200

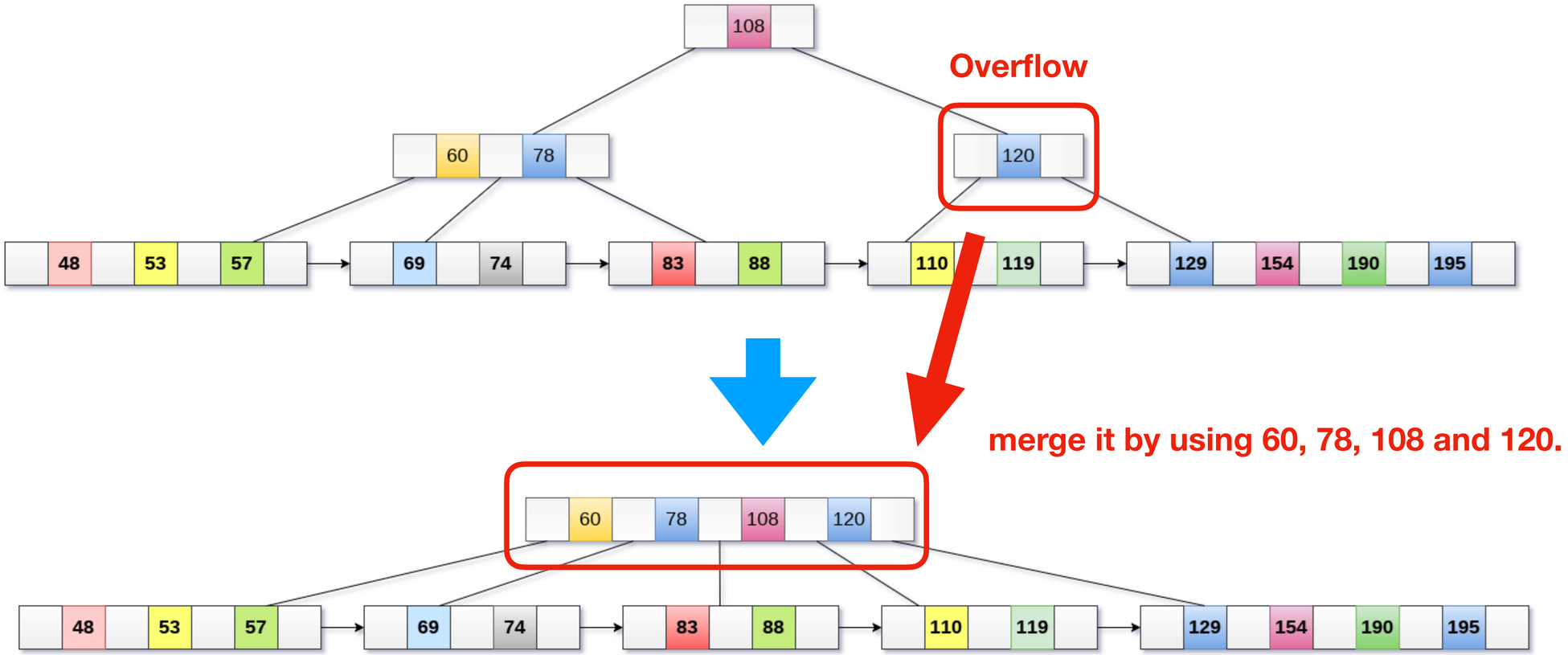
- 200 is present in the right sub-tree of 190, after 195. delete it.



Example : Deleting 200

- Merge the two nodes by using 195, 190, 154 and 129.



Example : Deleting 200

- Now, element 120 is the single element present in the node which is violating the B+ Tree properties. Therefore, we need to merge it by using 60, 78, 108 and 120.